

Case Study: Stabilizing an Inverted Pendulum on a Cart

Putting modeling, controllability, LQR, and simulation together on the classic benchmark.

A pole is hinged on a motor-driven cart; the goal is to keep the pole **upright** by moving the cart. The upright equilibrium is unstable – the pole falls on its own – so this problem exercises the full modern-control workflow. You will:

- linearize the cart-pole about the upright position,
- confirm the open loop is **unstable** but **controllable**,
- design a stabilizing state-feedback law with `lqr`, and
- simulate the recovery from a small initial tilt (before vs. after).

Builds on `State-Space/PolePlacement.m` and `State-Space/QuadraticOptimalRegulator.m`.

Run with `publish('InvertedPendulum.m')`, or step through with **Ctrl+Enter**.

Contents

- Linearized cart-pole model
- Before: the open loop is unstable
- Can we control it? Controllability check
- What we do: LQR state-feedback design
- After: recovery from an initial tilt
- What changed: open-loop vs. closed-loop pole angle
- Summary
- Try it yourself

Linearized cart-pole model

Parameters: cart mass M , pole mass m , half-length l , inertia I , cart friction b , gravity g . The state is $x = [p, \dot{p}, \phi, \dot{\phi}]^T$ (cart position and the pole angle from vertical). These are the standard linearized matrices.

```
M = 0.5; m = 0.2; b = 0.1; I = 0.006; g = 9.8; l = 0.3;
p = I*(M+m) + M*m*l^2;          % a common denominator

A = [0      1      0      0
     0 -(I+m*l^2)*b/p (m^2*g*l^2)/p 0
     0      0      0      1
     0 -(m*l*b)/p    m*g*l*(M+m)/p 0];
B = [0; (I+m*l^2)/p; 0; m*l/p];
C = [1 0 0 0; 0 0 1 0];          % measure cart position and pole angle
D = [0; 0];
sys = ss(A,B,C,D);
```

Before: the open loop is unstable

The pole sits at the top of its arc – a right-half-plane eigenvalue means any disturbance grows and the pole falls.

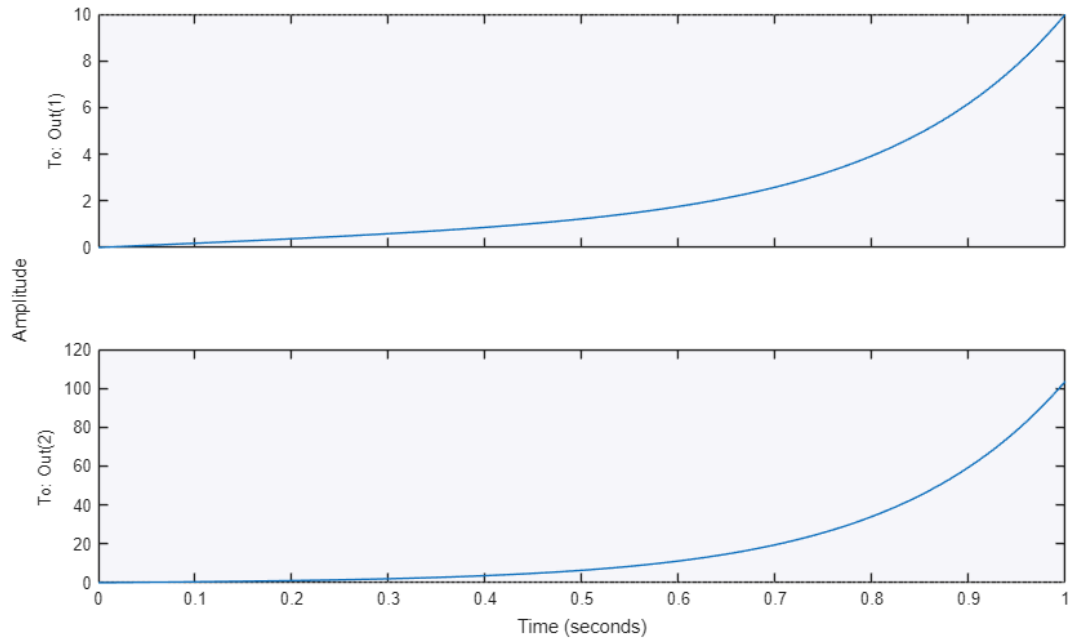
```
fprintf('Open-loop eigenvalues:\n'); disp(eig(A))
fprintf('Unstable (an eigenvalue with Re>0)? %d\n', any(real(eig(A))>0))

figure
impz(sys, 0:0.01:1)
title('Before: Open-Loop Response Diverges (Pole Falls)', 'Interpreter', 'latex', 'FontSize', 15)

Open-loop eigenvalues:
     0
-0.1428
-5.6041
 5.5651

Unstable (an eigenvalue with Re>0)? 1
```

Before: Open-Loop Response Diverges (Pole Falls)



Can we control it? Controllability check

A controller can only stabilize what it can reach. The pair (A, B) must be completely controllable.

```
fprintf('Controllability rank = %d (n = %d) -> controllable: %d\n', ...
    rank(ctrb(A,B)), size(A,1), rank(ctrb(A,B))==size(A,1))
```

```
Controllability rank = 4 (n = 4) -> controllable: 1
```

What we do: LQR state-feedback design

Choose the cost $J = \int (x^T Q x + u^T R u) dt$. We weight cart position and pole angle heavily (we care most about those), and pick R to keep the control force reasonable. `lqr` returns the stabilizing gain K .

```
Q = diag([5 0 20 0]);           % penalize position (1) and angle (3)
R = 0.1;
K = lqr(A,B,Q,R);
fprintf('\nLQR gain K = %s\n', mat2str(K,4))
fprintf('Closed-loop eigenvalues:\n'); disp(eig(A-B*K))
fprintf('All stable now? %d\n', all(real(eig(A-B*K))<0))
```

```
LQR gain K = [-7.071 -6.639 36.48 6.464]
Closed-loop eigenvalues:
-7.1565 + 4.5485i
-7.1565 - 4.5485i
-1.5894 + 1.3618i
-1.5894 - 1.3618i
```

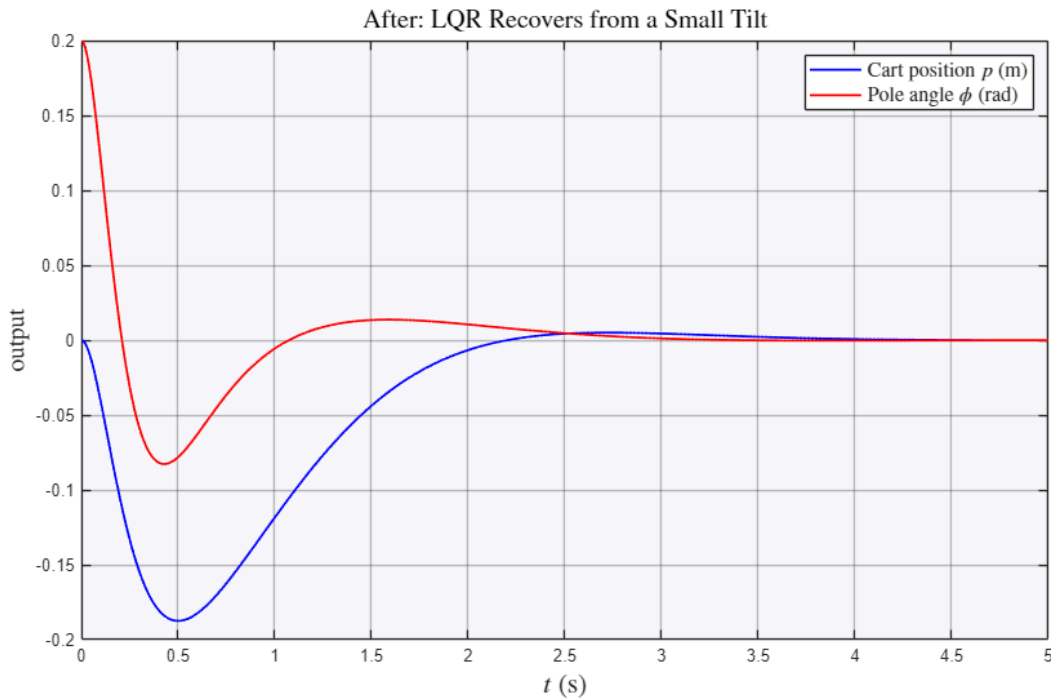
```
All stable now? 1
```

After: recovery from an initial tilt

Simulate the closed loop $\dot{x} = (A - BK)x$ from a small initial pole angle of about 11 degrees (0.2 rad). The controller drives both the pole angle and the cart position back to zero.

```
sys_cl = ss(A-B*K, B, C, D);
t = 0:0.01:5;
x0 = [0; 0; 0.2; 0];           % 0.2 rad initial tilt
[y,t] = initial(sys_cl, x0, t);

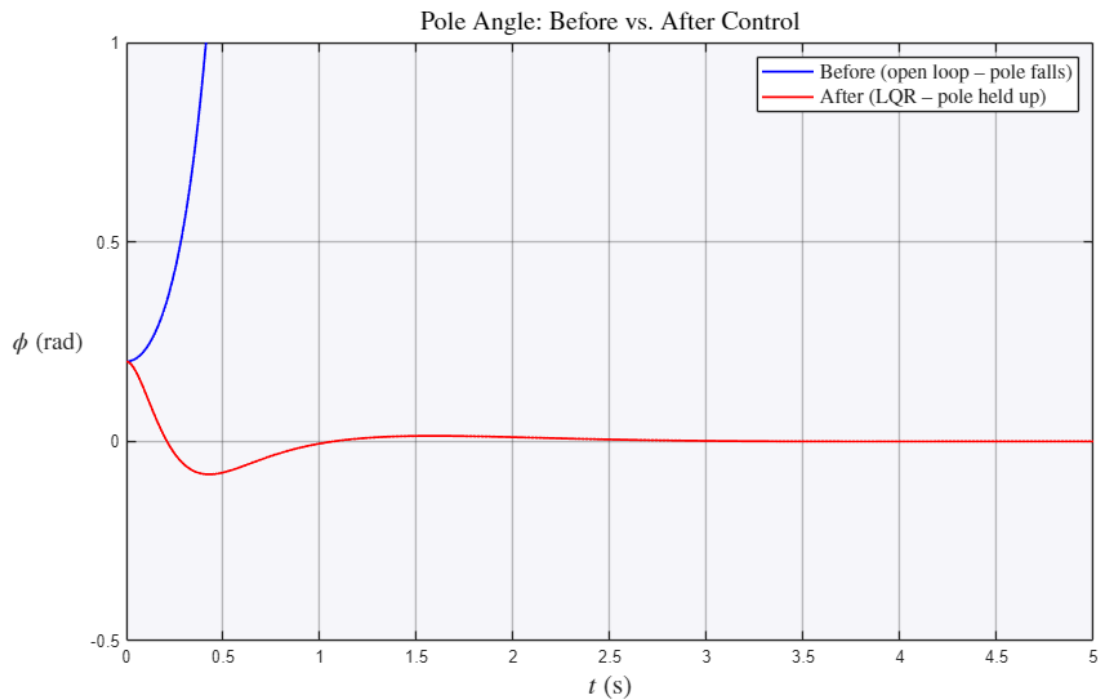
figure
plot(t, y(:,1),'b', t, y(:,2),'r','LineWidth',1.3)
grid on
legend('Cart position  $p$  (m)', 'Pole angle  $\phi$  (rad)','Interpreter','latex','FontSize',12)
title('After: LQR Recovers from a Small Tilt','Interpreter','latex','FontSize',15)
ylabel('output','Interpreter','latex','FontSize',14)
xlabel('t (s)','Interpreter','latex','FontSize',16)
```



What changed: open-loop vs. closed-loop pole angle

Side by side, the open-loop pole angle runs away while the controlled one is pulled back to vertical.

```
x_ol = initial(ss(A,zeros(4,1),C,D), x0, t);
figure
plot(t, x_ol(:,2),'b', t, y(:,2),'r','LineWidth',1.3)
grid on
ylim([-0.5 1])
legend('Before (open loop -- pole falls)', 'After (LQR -- pole held up)', ...
      'Interpreter','latex','FontSize',12)
title('Pole Angle: Before vs. After Control','Interpreter','latex','FontSize',15)
ylabel('phi (rad)','Interpreter','latex','FontSize',14); set(get(gca,'YLabel'),'Rotation',0)
xlabel('t (s)','Interpreter','latex','FontSize',16)
```



Summary

- The cart-pole linearizes to a 4-state model that is **unstable** but **controllable**.
- `lqr` delivers a single gain K that stabilizes it; the Q/R weights trade cart travel and pole angle against control force.
- From a tilt the closed loop returns the pole to vertical and the cart to its origin – the whole pipeline in one example.

Try it: increase the angle weight in Q and watch the pole snap upright faster (at the cost of more cart motion and control force).

Try it yourself

- Raise R and watch the controller get gentler – less force, more cart travel, a slower recovery.
- Stop penalizing cart position (zero $Q(1,1)$) and notice the pole still balances while the cart is free to drift.